

Database Systems (CS2005)

Week – 10

Instructor: **Basit Ali**

CHAPTER 14

Basics of Functional Dependencies and Normalization for Relational Databases

Normalization | | Data Normalization

- Normalization is the process of reorganizing/restructuring data in a database with a series of so called normal-forms, so that it meets two basic requirements:
 - (1) There is no redundancy of data (all data is stored in only one place), and
 - (2) data dependencies are logical (all related data items are stored together).
- Normalization is important for many reasons, but chiefly because it allows databases to take up as little disk space as possible, resulting in increased performance.

Redundant Information in Tuples and Update Anomalies

- Information is stored redundantly
 - Wastes storage
 - Causes problems with update anomalies
 - Insertion anomalies
 - Deletion anomalies
 - Modification anomalies

EXAMPLE OF AN UPDATE ANOMALY

- Consider the relation:
 - EMP_PROJ(Emp#, Proj#, Ename, Pname, No_hours)
- Update Anomaly:
 - Changing the name of project number P1 from “Billing” to “Customer-Accounting” may cause this update to be made for all 100 employees working on project P1.

EXAMPLE OF AN INSERT ANOMALY

- Consider the relation:
 - EMP_PROJ(Emp#, Proj#, Ename, Pname, No_hours)
- Insert Anomaly:
 - Cannot insert a project unless an employee is assigned to it.
- Conversely
 - Cannot insert an employee unless an he/she is assigned to a project.

EXAMPLE OF A DELETE ANOMALY

- Consider the relation:
 - EMP_PROJ(Emp#, Proj#, Ename, Pname, No_hours)
- Delete Anomaly:
 - When a project is deleted, it will result in deleting all the employees who work on that project.
 - Alternately, if an employee is the sole employee on a project, deleting that employee would result in deleting the corresponding project.

Figure 14.4
Sample states for
EMP_DEPT and
EMP_PROJ
resulting from
applying NATURAL
JOIN to the
relations in Figure
14.2. These may
be stored as base
relations for
performance
reasons.

EMP_DEPT					Redundancy	
Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5	Research	333445555
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4	Administration	987654321
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4	Administration	987654321
Narayan, Ramesh K.	666884444	1962-09-15	975 FireOak, Humble, TX	5	Research	333445555
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5	Research	333445555
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4	Administration	987654321
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1	Headquarters	888665555

EMP_PROJ			Redundancy		Redundancy	
Ssn	Pnumber	Hours	Ename	Pname	Plocation	
123456789	1	32.5	Smith, John B.	ProductX	Bellaire	
123456789	2	7.5	Smith, John B.	ProductY	Sugarland	
666884444	3	40.0	Narayan, Ramesh K.	ProductZ	Houston	
453453453	1	20.0	English, Joyce A.	ProductX	Bellaire	
453453453	2	20.0	English, Joyce A.	ProductY	Sugarland	
333445555	2	10.0	Wong, Franklin T.	ProductY	Sugarland	
333445555	3	10.0	Wong, Franklin T.	ProductZ	Houston	
333445555	10	10.0	Wong, Franklin T.	Computerization	Stafford	
333445555	20	10.0	Wong, Franklin T.	Reorganization	Houston	
999887777	30	30.0	Zelaya, Alicia J.	Newbenefits	Stafford	
999887777	10	10.0	Zelaya, Alicia J.	Computerization	Stafford	
987987987	10	35.0	Jabbar, Ahmad V.	Computerization	Stafford	
987987987	30	5.0	Jabbar, Ahmad V.	Newbenefits	Stafford	
987654321	30	20.0	Wallace, Jennifer S.	Newbenefits	Stafford	
987654321	20	15.0	Wallace, Jennifer S.	Reorganization	Houston	
888665555	20	Null	Borg, James E.	Reorganization	Houston	

Guideline for Redundant Information in Tuples and Update Anomalies

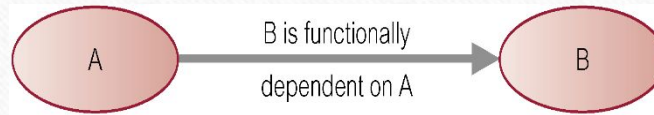
- GUIDELINE
 - Design a schema that does not suffer from the insertion, deletion and update anomalies.
 - If there are any anomalies present, then note them so that applications can be made to take them into account.

Functional Dependencies

- Functional dependencies (FDs)
 - Are used to specify *formal measures* of the "goodness" of relational designs
 - And keys are used to define **normal forms** for relations
 - Are **constraints** that are derived from the *meaning* and *interrelationships* of the data attributes
- A set of attributes X *functionally determines* a set of attributes Y if the value of X determines a unique value for Y

Defining Functional Dependencies

- If A and B are attributes of relation R , B is functionally dependent on A (denoted $A \rightarrow B$), if each value of A in R is associated with exactly one value of B in R .



- $X \rightarrow Y$ holds if whenever two tuples have the same value for X , they *must have* the same value for Y
 - For any two tuples $t1$ and $t2$ in any relation instance $r(R)$: If $t1[X]=t2[X]$, *then* $t1[Y]=t2[Y]$
- $X \rightarrow Y$ in R specifies a *constraint* on all relation instances $r(R)$
- Written as $X \rightarrow Y$; can be displayed graphically on a relation schema as in Figures. (denoted by the arrow: $\rightarrow$).
- FDs are derived from the real-world constraints on the attributes

Examples of FD constraints

- Social security number determines employee name
 - $SSN \rightarrow ENAME$
- Project number determines project name and location
 - $PNUMBER \rightarrow \{PNAME, PLOCATION\}$
- Employee ssn and project number determines the hours per week that the employee works on the project
 - $\{SSN, PNUMBER\} \rightarrow HOURS$

Important Definitions

- **Determinant**

- Refers to the attribute, or group of attributes, on the left-hand side of the arrow of a functional dependency.

- **Full Functional dependency:**

- Indicates that if A and B are attributes of a relation, B is fully functionally dependent on A if B is functionally dependent on A, but not on any proper subset of A.
 - $\{\text{StaffNo}, \text{StaffName}\} \twoheadrightarrow \text{BranchNo}$
 - $\text{StaffNo} \twoheadrightarrow \text{BranchNo}$

- **Transitive Dependency**

- A condition where A, B, and C are attributes of a relation such that if $A \rightarrow B$ and $B \rightarrow C$, then C is

Example Transitive Dependency

StaffBranch

staffNo	sName	position	salary	branchNo	bAddress
SL21	John White	Manager	30000	B005	22 Deer Rd, London
SG37	Ann Beech	Assistant	12000	B003	163 Main St, Glasgow
SG14	David Ford	Supervisor	18000	B003	163 Main St, Glasgow
SA9	Mary Howe	Assistant	9000	B007	16 Argyll St, Aberdeen
SG5	Susan Brand	Manager	24000	B003	163 Main St, Glasgow
SL41	Julie Lee	Assistant	9000	B005	22 Deer Rd, London

- Consider functional dependencies in the StaffBranch relation

$\text{staffNo} \rightarrow \text{sName, position, salary, branchNo, bAddress}$

$\text{branchNo} \rightarrow \text{bAddress}$

- Transitive dependency,

- $\text{branchNo} \rightarrow \text{bAddress}$ exists on staffNo via branchNo

Normalization of Relations

- **Normalization:**
 - The process of decomposing unsatisfactory "bad" relations by breaking up their attributes into smaller relations
- **Normal form:**
 - Condition using keys and FDs of a relation to certify whether a relation schema is in a particular normal form

Normalization of Relations

- 2NF, 3NF, BCNF
 - based on keys and FDs of a relation schema
- 4NF
 - based on keys, multi-valued dependencies : MVDs;
- 5NF
 - based on keys, join dependencies : JDs
- Additional properties may be needed to ensure a good relational design (lossless join, dependency preservation)

Practical Use of Normal Forms

- **Normalization** is carried out in practice so that the resulting designs are of high quality and meet the desirable properties
- The practical utility of these normal forms becomes questionable when the constraints on which they are based are *hard to understand* or to *detect*
- The database designers *need not* normalize to the highest possible normal form
 - (usually up to 3NF and BCNF. 4NF rarely used in practice.)
- **Denormalization:**
 - The process of storing the join of higher normal form relations as a base relation—which is in a lower normal

Definitions of Keys and Attributes Participating in Keys

- A **superkey** of a relation schema $R = \{A_1, A_2, \dots, A_n\}$ is a set of attributes S *subset-of* R with the property that no two tuples t_1 and t_2 in any legal relation state r of R will have $t_1[S] = t_2[S]$
- A **key** K is a **superkey** with the *additional property* that removal of any attribute from K will cause K not to be a superkey any more.

First Normal Form

- Disallows
 - composite attributes
 - multivalued attributes (no repeating group)

• nested relations; attributes whose values for an *individual tuple* are non-atomic (Composite)

title	author_list	date	keyword_list
		day month year	
salesplan	{Smith, Jones}	1 April 89	{profit, strategy}
stat. report	{Jones, Frick}	17 July 94	{profit, personnel}

- Considered to be part of the definition of a relation
- Most RDBMSs allow only those relations to be defined that are in First Normal Form

Figure 14.9
 Normalization into 1NF. (a) A
 relation schema that is not in 1NF.
 (b) Sample state of relation
 DEPARTMENT. (c) 1NF version of
 the same relation with
 redundancy.

(a)

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	Dlocations

(b)

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

(c)

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	<u>Dlocation</u>
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

(a)

EMP_PROJ

Ssn	Ename	Projs	
		Pnumber	Hours

(b)

EMP_PROJ

Ssn	Ename	Pnumber	Hours
123456789	Smith, John B.	1	32.5
		2	7.5
666884444	Narayan, Ramesh K.	3	40.0
453453453	English, Joyce A.	1	20.0
		2	20.0
333445555	Wong, Franklin T.	2	10.0
		3	10.0
		10	10.0
		20	10.0
999887777	Zelaya, Alicia J.	30	30.0
		10	10.0
987987987	Jabbar, Ahmad V.	10	35.0
		30	5.0
987654321	Wallace, Jennifer S.	30	20.0
		20	15.0
888665555	Borg, James E.	20	NULL

(c)

EMP_PROJ1

<u>Ssn</u>	Ename
------------	-------

EMP_PROJ2

<u>Ssn</u>	<u>Pnumber</u>	Hours
------------	----------------	-------

Figure 14.10

Normalizing nested relations into 1NF.

(a) Schema of the EMP_PROJ relation with a nested relation attribute PROJS.

(b) Sample extension of the EMP_PROJ relation showing nested relations within each tuple. (c) Decomposition of EMP_PROJ into relations EMP_PROJ1 and EMP_PROJ2 by propagating the primary key.

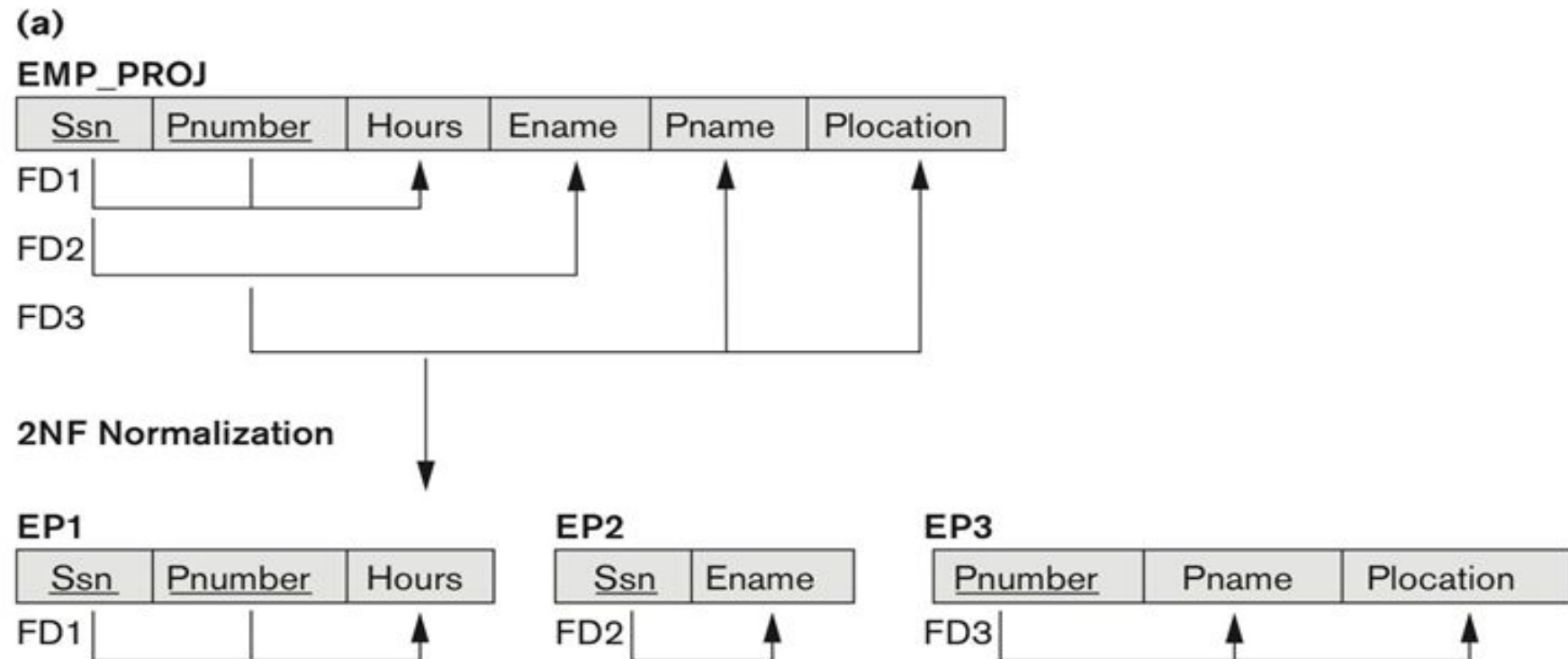
Second Normal Form

- Uses the concepts of **FDs, primary key**
- Definitions
 - **Prime attribute:** An attribute that is member of the primary key K
 - **Full functional dependency:** a FD $Y \rightarrow Z$ where removal of any attribute from Y means the FD does not hold any more
- Examples:
 - $\{SSN, PNUMBER\} \rightarrow HOURS$ is a full FD since neither $SSN \rightarrow HOURS$ nor $PNUMBER \rightarrow HOURS$ hold
 - $\{SSN, PNUMBER\} \rightarrow ENAME$ is not a full FD (it is called a partial dependency) since $SSN \rightarrow ENAME$ also holds

Second Normal Form

- A relation schema R is in **second normal form (2NF)** if every non-prime attribute A in R is fully functionally dependent on the primary key
- R can be decomposed into 2NF relations via the process of 2NF normalization or “second normalization”

Figure 14.11
Normalizing into 2NF and 3NF.
(a) Normalizing EMP_PROJ into 2NF relations.



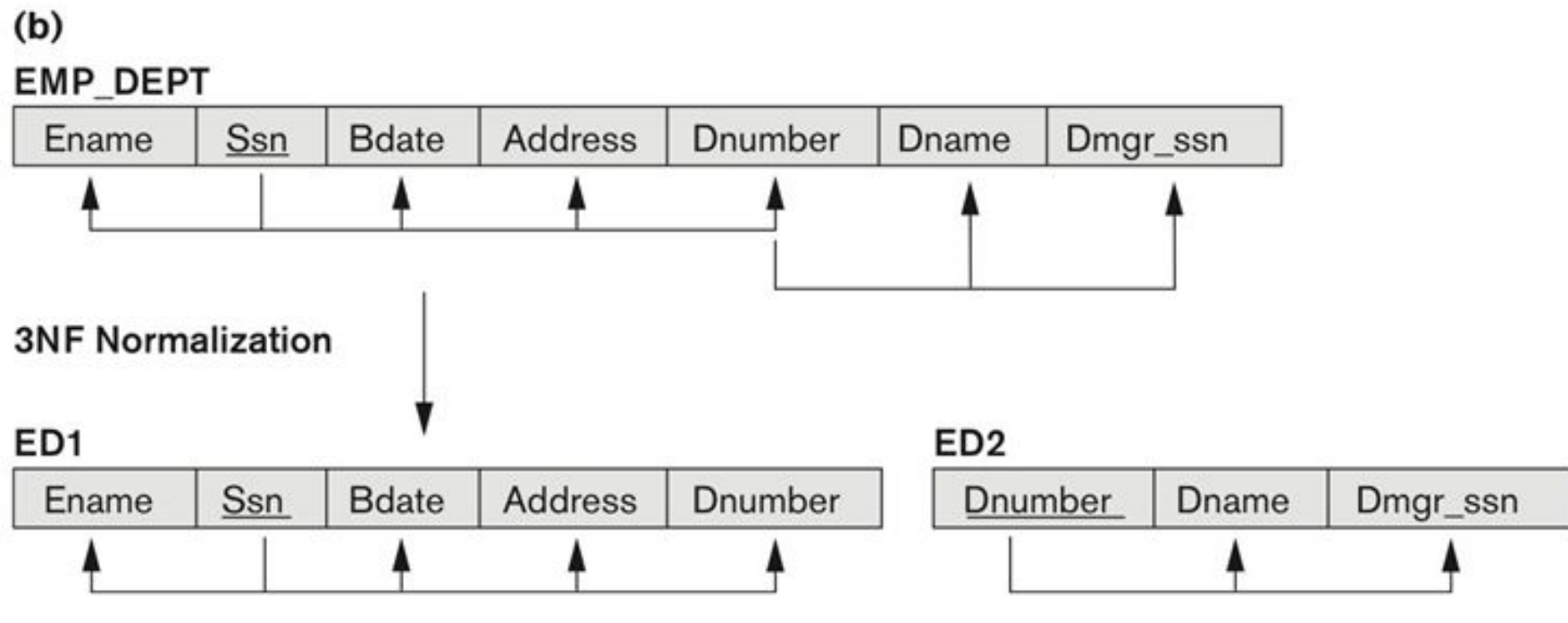
Third Normal Form

- Definition:
 - **Transitive functional dependency:** a FD $X \rightarrow Z$ that can be derived from two FDs $X \rightarrow Y$ and $Y \rightarrow Z$
- Examples:
 - $SSN \rightarrow DMGRSSN$ is a **transitive** FD
 - Since $SSN \rightarrow DNUMBER$ and $DNUMBER \rightarrow DMGRSSN$ hold
 - $SSN \rightarrow ENAME$ is **non-transitive**
 - Since there is no set of attributes X where $SSN \rightarrow X$ and $X \rightarrow ENAME$

Third Normal Form

- A relation schema R is in **third normal form (3NF)** if it is in 2NF *and* no non-prime attribute A in R is transitively dependent on the primary key
- R can be decomposed into 3NF relations via the process of 3NF normalization
- NOTE:
 - In $X \rightarrow Y$ and $Y \rightarrow Z$, with X as the primary key, we consider this a problem only if Y is not a candidate key.
 - When Y is a candidate key, there is no problem with the transitive dependency .
 - E.g., Consider EMP (SSN, Emp#, Salary).
 - Here, $SSN \rightarrow Emp\# \rightarrow Salary$ and Emp# is a candidate key.

Figure 14.11
Normalizing into 3NF.
(b) Normalizing EMP_DEPT into 3NF relations.



Step-by-Step Normalization

StudentID	StudentName	Course1	Course2	Course3	DeptName	DeptLocation
1	Ali	DBMS	OOP	NULL	CS	Block A
2	Sara	AI	ML	DBMS	CS	Block A
3	Hamza	Stats	NULL	NULL	Math	Block B

1 NF

StudentID	StudentName	Course	DeptName	DeptLocation
1	Ali	DBMS	CS	Block A
1	Ali	OOP	CS	Block A
2	Sara	AI	CS	Block A
2	Sara	ML	CS	Block A
2	Sara	DBMS	CS	Block A
3	Hamza	Stats	Math	Block B

2 NF

1. Student Table

StudentID	StudentName	DeptName	DeptLocation
1	Ali	CS	Block A
2	Sara	CS	Block A
3	Hamza	Math	Block B

2. Enrollment Table

StudentID	Course
1	DBMS
1	OOP
2	AI
2	ML
2	DBMS
3	Stats

3 NF

1. Student Table

StudentID	StudentName	DeptName
1	Ali	CS
2	Sara	CS
3	Hamza	Math

2. Department Table

DeptName	DeptLocation
CS	Block A
Math	Block B

3. Enrollment Table (unchanged)

StudentID	Course
1	DBMS
1	OOP
2	AI
2	ML
2	DBMS
3	Stats

Exercise

StudentID	StudentName	CourseDetails
1	Ahmed	101, Database, Prof. Ali
2	Sana	102, Networks, Prof. Ahmed
3	Hina	101, Database, Prof. Ali
4	Ali	103, Algorithms, Prof. Sara

(BCNF or 3.5) Normal Form

- BCNF (Boyce-Codd Normal Form) is an advanced version of the Third Normal Form (3NF).
- A table is in BCNF if it is in 3NF and every determinant is a candidate key.
- This means that for any functional dependency $(A \rightarrow B)$, A must be a superkey (a set of attributes that uniquely identifies a row in a table).

Example

- Consider the following table *EmployeeAssignments*

EmployeeID	Project	Manager
E01	P1	M1
E01	P2	M2
E02	P1	M1
E02	P3	M3

- Functional Dependencies:
 1. EmployeeID, Project $\rightarrow$ Manager (An employee is assigned to a specific manager for a project.)
 2. Project $\rightarrow$ Manager (A project is always managed by the same manager.)

Example (cont..)

- Problem?
 - The table is in 3NF, but it's not in BCNF because Project is not a superkey, yet it determines Manager
- Solution?
 - To bring it to BCNF, we decompose the table into two tables:
 - **Projects**
 - **EmployeeAssignments**

Example (cont..)

- Projects table (stores the relationship between Project and Manager):

Project	Manager
P1	M1
P2	M2
P3	M3

Example (cont..)

- EmployeeAssignments table (assigns employees to projects):

EmployeeID	Project
E01	P1
E01	P2
E02	P1
E02	P3

Example (cont..)

- Now both tables are in BCNF because:
 - In the **Projects** table, the determinant **Project** is a candidate key.
 - In the **EmployeeAssignments** table, the determinant **EmployeeID**, **Project** is a candidate key.